Association Rule Mining

4DT910/4DV510: Data Mining

Association Rule Mining: Finding Things That Occur Together

Goal: Discover interesting relationships hidden in large datasets

Association Rule Mining finds patterns of the form:

When X occurs, Y also tends to occur: $X \rightarrow Y$

Domain	Example association rule
Retail	{pasta, tomato sauce} → {parmesan}
Healthcare	{diagnosis, laboratory finding} → {medication}
Web	{visited page A, page B} → {page C}
Banking	{service A, service B} → {service C}
Apps/services	{uses feature A, feature B} → {uses feature C}

The Classic Example: Market Basket Analysis

Setting: A supermarket has millions of transactions (baskets).

Each transaction contains a set of items purchased by a customer.

Example of transactions:

Transaction ID	Items in basket
T1	{milk, bread, butter}
T2	{bread, beer, chips}
T3	{milk, bread, eggs}
T4	{pasta, tomato sauce, parmesan}
T5	{milk, bread, pasta, tomato sauce}
…	…

Business question:

Which products are purchased together more often than expected?

Example association rule:

{pasta, tomato sauce} → {parmesan}

Possible business uses:

- Product recommendations and cross-selling
- Promotions and targeted marketing
- Product bundling and catalog design
- Store layout and shelf placement
- Demand forecasting and inventory planning



Chen, Y.L., Tang, K., Shen, R.J. and Hu, Y.H., 2005. Market basket analysis in a multiple store environment. *Decision support systems*, *40*(2), pp.339-354.

Beyond Shopping Baskects

Not all datasets naturally contain transactions

Data preparation matters

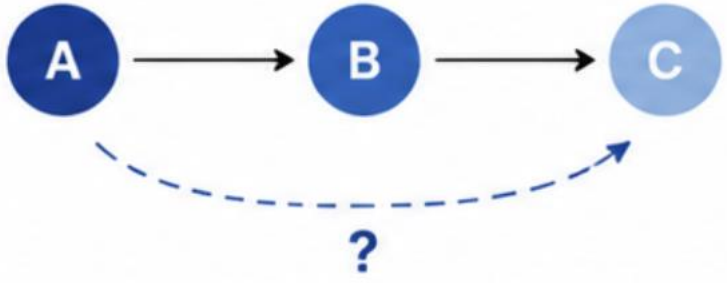
Define the **transaction** with domain experts

Decide which **items** belong to it

Different design choices → **different association rules**

Data unit (example)	**Possible items**	**How a transaction** can be defined (examples)	**Real applications** (examples from research)
Customer basket	Products purchased	One **checkout** = one transaction	Market basket analysis in retail stores *Chen et al., Decision Support Systems, 2005*
Patient record	Diagnoses, medications, laboratory findings, procedures	• One **patient** = one transaction • One hospital **admission** = one transaction • One **visit** = one transaction • **Time window** (e.g., 30 days) = one transaction	Discover medication–diagnosis associations; identify possible missing diagnoses *Wright et al., J. Biomed. Inform., 2010; JAMIA, 2013*
Document	Words, concepts, keywords, metadata	One **document** (or section) = one transaction	Analyze historical archives and find relationships between topics/keywords *Esteva et al., J. Assoc. Inf. Sci. Technol., 2014*
Web session	Visited pages, clicked items	One user **session** (from start to end) = one transaction	Web usage mining for navigation patterns and recommendations *Mobasher et al., Web Intelligence, 2002*
Computer / Time window	Log events, alerts, errors, warnings	Events occurring within a **time window** (e.g., 5 min, 1 hour) = one transaction	Discover co-occurring events in system logs for fault diagnosis and cyberseactiry *Kumar et al., Expert Systems with Applications, 2021; Hamdi et al., Computers & Security, 2020*

Example



Example (illustrative)

Known relationships	Potential new relationship
ACE inhibitors ↔ Cough Cough ↔ Angioedema	ACE inhibitors ↔ Angioedema *(predicted)*

How it was tested

Learn relationships from older literature (MEDLINE + UMLS).

↓

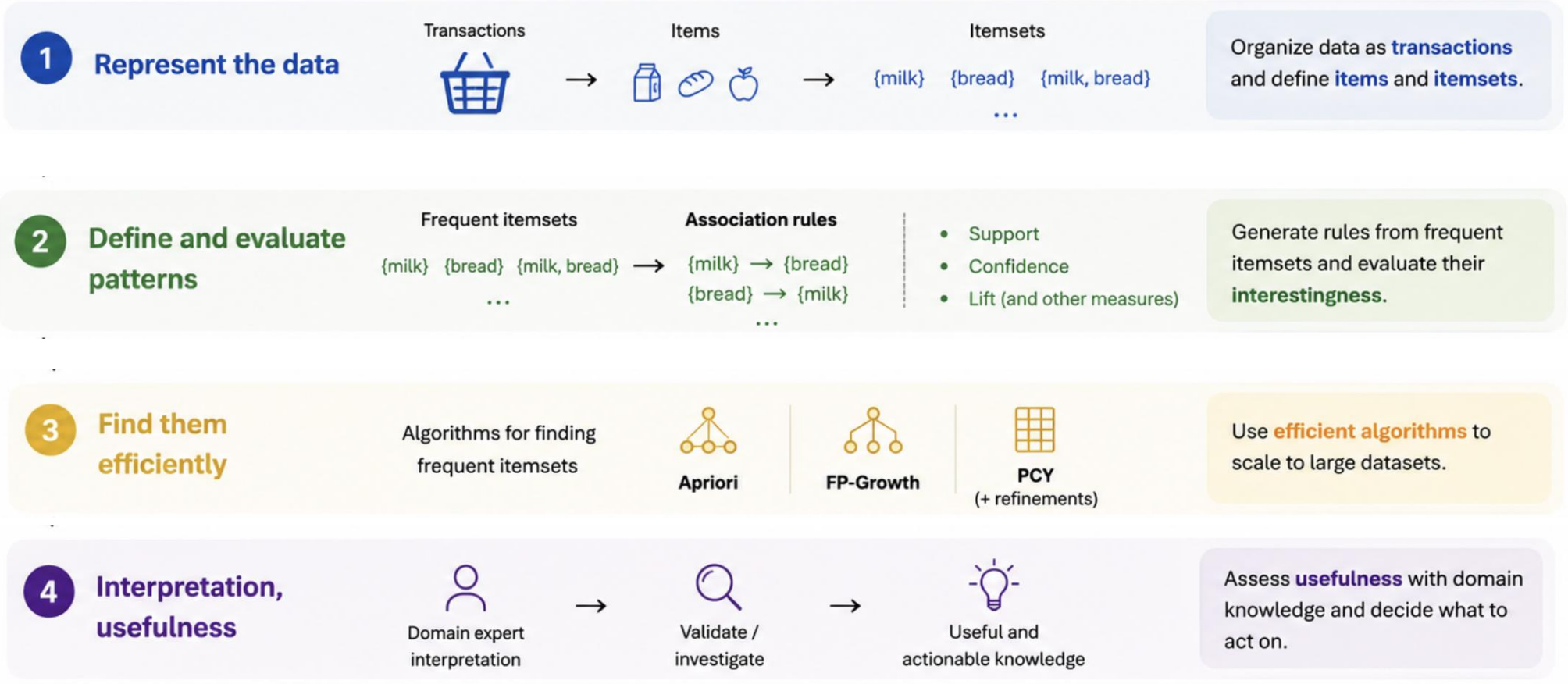
Examine whether proposed A–C relationships later appear in newer scientific literature.

↓

Result:
Predicted future relationships **significantly better** than random prediction.

Hristovski et al. (2001). Supporting discovery in medicine by association rule mining in Medline and UMLS.
J. Biomedical Informatics, *34*(2), 90–98.
https://pubmed.ncbi.nlm.nih.gov/11604946/

Association Rule Mining Outline



Frequent Itemsets

- **Goal**: Find sets of items that appear together "frequently" in transactions (e.g., baskets)
- **Itemset**
  - A collection of one or more items $X = \{\text{Apples, Milk}\}$
- **Support count**
  - Frequency of occurrences of an itemset $\sigma(X) = |\{T_i \mid X \subseteq T_i\}|$
- **Support**
  - Fraction of transactions that contain an itemset $\text{support}(X) = \frac{\sigma(X)}{N}$
- **Frequent Itemset**
  - An itemset whose support is greater than or equal to a *minimum support threshold*

$$\text{support}(X) \geq \text{minsup}$$

Example

TID	Items in transaction
1	Apples, Bananas, Milk
2	Apples, Bread, Cheese
3	Bananas, Bread, Milk
4	Apples, Bananas, Bread
5	Apples, Milk
6	Bread, Cheese, Milk

Example 1

Itemset: **{Apples, Milk}**

Appears in transactions: **1, 5**

1) Support count (σ)

$\sigma(X) = |\{T_i \mid X \subseteq T_i\}|$

$\sigma(\{\text{Apples, Milk}\}) = |\{T_1, T_5\}| = 2$

2) Support (s)

$s(X) = \frac{\sigma(X)}{N}$

N = total number of transactions = 6

$s(\{\text{Apples, Milk}\}) = 2 / 6 = 0.333 = 33.3\%$

3) Minimum support threshold (minsup)

minsup = **30% = 0.30**

4) Is the itemset frequent?

$s(X) \geq \text{minsup}$

$33.3\% \geq 30\% \Rightarrow$ **Yes**

{Apples, Milk} is a frequent itemset ✓

Example 2

Itemset: **{Apples, Cheese}**

Appears in transactions: **2**

1) Support count (σ)

$\sigma(X) = |\{T_i \mid X \subseteq T_i\}|$

$\sigma(\{\text{Apples, Cheese}\}) = |\{T_2\}| = 1$

2) Support (s)

$s(X) = \frac{\sigma(X)}{N}$

N = total number of transactions = 6

$s(\{\text{Apples, Cheese}\}) = 1 / 6 = 0.167 = 16.7\%$

3) Minimum support threshold (minsup)

minsup = **30% = 0.30**

4) Is the itemset frequent?

$s(X) \geq \text{minsup}$

$16.7\% < 30\} \Rightarrow$ **No**

{Apples, Cheese} is NOT frequent ✗

σ(X) – **support count**: number of transactions that contain itemset X

s(X) – **support**: fraction of all transactions that contain X

minsup – **minimum support threshold**: the minimum support required for an itemset to be considered frequent

Association Rules

- **Association Rule**
  - An implication expression of the form X → Y, where X and Y are itemsets
  - "*if a transaction contains all of items in X then it is likely to contain Y items*"
- **Rule Evaluation Metrics**
  - Support (s):

$$\text{support}(X) = \frac{\sigma(X)}{N}$$

  - Confidence (c):
    - Measure how often items in Y appear in transactions that contain X.

$$\text{conf}(X \rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

Example

Transactions ($N = 6$)

TID	Items in transaction
1	Coffee, Sugar
2	Coffee, Sugar
3	Coffee, Sugar
4	Coffee
5	Coffee
6	Tea

Support counts

- Coffee appears in **5** transactions
- Sugar appears in **3** transactions
- Coffee and Sugar appear together in **3** transactions

Supports

- support(Coffee) = 5/6 = 0.833 (83.3%)
- support(Sugar) = 3/6 = 0.500 (50.0%)
- support(Coffee, Sugar) = 3/6 = 0.500 (50.0%)

Rule 1: Coffee → Sugar

Confidence tells us: When Coffee is bought, how often is Sugar also bought?

1) Formula

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

2) Substitute

$$= \frac{support(Coffee,\ Sugar)}{support(Coffee)}$$

3) Use support values

$$= \frac{3/6}{5/6} = \frac{3}{5} = 0.60 = 60\%$$

Confidence(Coffee → Sugar) = 60%

Of transactions containing Coffee, 60% also contain Sugar.

Rule 2: Sugar → Coffee

Confidence tells us: When Sugar is bought, how often is Coffee also bought?

1) Formula

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

2) Substitute

$$= \frac{support(Sugar,\ Coffee)}{support(Sugar)}$$

3) Use support values

$$= \frac{3/6}{3/6} = \frac{3}{3} = 1.00 = 100\%$$

Confidence(Sugar → Coffee) = 100%

Of transactions containing Sugar, 100% also contain Coffee.

Summary:

- **Support is symmetric:** support(Coffee, Sugar) = support(Sugar, Coffee) = 3/6 = 50%
- **Confidence is directional:** Coffee → Sugar (60%) ≠ Sugar → Coffee (100%)

Lift and Interest

Definitions

Lift measures the strength of association as a ratio.

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)} = \frac{\text{Support}(X \cup Y)}{\text{Support}(X) \times \text{Support}(Y)}$$

Interest measures the strength of association as a difference.

$$\text{Interest}(X \to Y) = \text{Confidence}(X \to Y) - \text{Support}(Y) = P(Y|X) - P(Y)$$

Support(Z) = fraction (probability) of transactions that contain Z.

Example data (N = 6 transactions)

TID	Items in transaction
1	Coffee, Sugar
2	Coffee, Sugar
3	Coffee, Sugar
4	Coffee
5	Coffee
6	Tea

Supports

- Support(Coffee) = 5/6 = 0.833
- Support(Sugar) = 3/6 = 0.500
- Support(Coffee, Sugar) = 3/6 = 0.500

Rule: Coffee → Sugar

- Confidence(Coffee → Sugar) = 3/5 = 0.60 (60%)
- Support(Sugar) = 3/6 = 0.50

Interpretation

Lift > 1	/	Interest > 0	→ positive association
= 1	/	= 0	→ independent
< 1	/	< 0	→ negative association

Compute Lift and Interest

Lift(Coffee → Sugar)

= Confidence / Support(Sugar)

= 0.60 / 0.50 = 1.20

→ Lift > 1 (positive association)

Interest(Coffee → Sugar)

= Confidence − Support(Sugar)

= 0.60 − 0.50 = 0.10

→ Interest > 0 (positive association)

Why high confidence does NOT always mean an interesting rule

Example: Milk is purchased very often

- Support(Milk) = 0.80 (80% of customers buy milk)

Rule	Confidence	Lift	Interest
Bread → Milk	0.90	1.13	+0.10
Cereal → Milk	0.88	1.10	+0.08
Cookies → Milk	0.87	1.09	+0.07

Interpretation

- All rules have high confidence (≈ 0.9) because Milk is bought very often.
- But Lift is only slightly above 1 and Interest is close to 0.
- These are weakly associated rules.

Compare: a strong rule

Fruit → Yogurt

- Support(Yogurt) = 0.20
- Confidence = 0.60

Lift	Interest
3.00	+0.40

Strong positive association!

Summary: Use **Lift** or **Interest** to judge whether a rule is stronger (positive), weaker (negative), or independent. High confidence alone can be misleading when *Y* is very common.

Finding Association Rules

- **Task**: Find all association rules X→Y with support *support(X∪Y)≥minsup* and *confidence(X→Y)≥minconf*

1 Generate candidate itemsets

For each possible non-empty itemset $F \subseteq I$:

- Compute $support(F)$.
- If $support(F) \geq minsup$:
  - Mark F as a frequent itemset.

2 Generate association rules

- For every pair (X, Y) with $X \subset F$, compute confidence using

$$confidence(X \rightarrow Y) = \frac{support(F)}{support(X)}.$$

- Output the rule if confidence $\geq minconf$.
- Simple but may compute many rules that will be rejected.

Variant 2: Pruning (build larger rules from smaller ones)

- **Observation** (downward-closure for confidence):

If $confidence(X \rightarrow Y) < minconf$,
then for any $X' \supset X$ and $Y' = F \setminus X'$,
$confidence(X' \rightarrow Y') \leq$ confidence$(X \rightarrow Y)$.

- Therefore, if a rule is below $minconf$, all its more-specific (larger antecedent) rules from the same F can be skipped.
- Explore from smaller X to larger X and prune early
  ⇒ much fewer computations.

3 Compact the output (post-processing)

To reduce the number of rules, we can post-process and only output:

- **Maximal frequent itemsets**
  No immediate superset is frequent.
  Formally: F is *maximal* if there is no $G \supset F$ such that $support(G) \geq minsup$.
  → Gives more pruning.
- **Closed itemsets**
  No immediate superset has the same support count (> 0).
  Formally: F is *closed* if for any $G \supset F$, $support(G) < support(F)$.
  → Retains all frequent patterns without redundancy.

Computationally prohibitive!

Interpretation Problem!

Computational Complexity

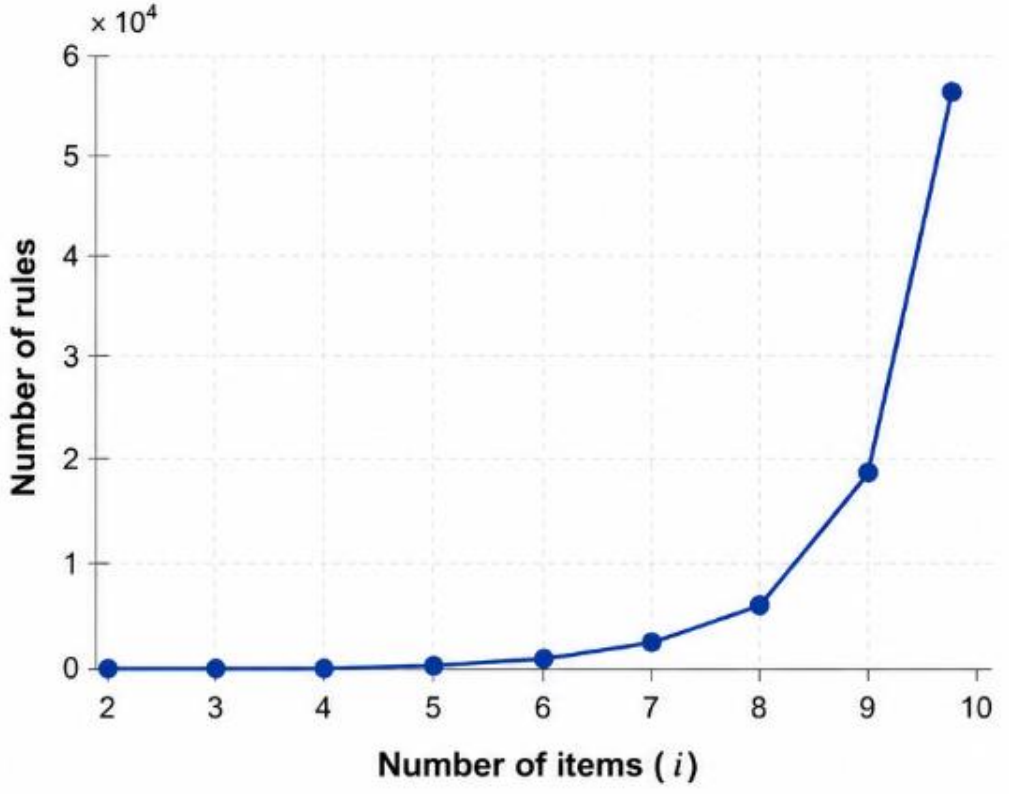
- Given *i* unique items:
  - Total number of itemsets = 2^i
  - Total number of possible association rules:

$$R = \sum_{k=1}^{i-1} \left[\binom{i}{k} \times \sum_{j=1}^{i-k} \binom{i-k}{j} \right]$$

$$= 3^i - 2^{i+1} + 1$$

Example:

If $i = 6$, $R =$ **602 rules**



We need efficient methods to avoid exhaustive search

Frequent Itemset Generation Strategies

1 Reduce the number of candidates (M)

- Avoid testing itemsets that cannot possibly be frequent.

Example algorithms

- Apriori (level-wise, pruning)
- FP-Growth (no candidate generation)
- Eclat (uses intersections)
- CHARM / MAFIA (closed itemsets)

2 Reduce the number of transactions (N)

- Ignore transactions that cannot contain the current larger itemsets.

Example algorithms

- DHP (Dynamic Hash Pruning)
- Vertical-based algorithms (e.g., Eclat, CHARM)
- Diffset / Tidset approaches

3 Reduce the number of comparisons ($N \times M$)

- Use efficient data structures to avoid checking every candidate against every transaction.

Example algorithms / techniques

- FP-Growth (FP-tree)
- Eclat (tidset intersections)
- Bitmap / Bitset approaches
- Hash-based counting

Apriori Algorithm

Notation

C_k : candidate k-itemsets

L_k : frequent k-itemsets

Algorithm

1. **Initialization**
   Find L_1 = frequent 1-itemsets
   (i.e., all items with support $\geq minsup$).
2. **Set**
   $k = 1$
3. **Repeat while L_k is not empty**

3.1 Candidate Generation	Generate C_{k+1} from L_k by joining itemsets of size k.
3.2 Candidate Pruning	Prune candidates in C_{k+1} if any of their k-item subsets is not in L_k.
3.3 Support Counting	Scan the transaction database and count the support of each candidate in C_{k+1}.
3.4 Keep Frequent Candidates	Keep only those candidates that are frequent: $L_{k+1} = \{X \in C_{k+1} \mid support(X) \geq minsup\}$

4. **Increase k**
   $k = k + 1$

Apriori Property (Downward Closure)

If an itemset is frequent, all of its subsets must also be frequent.

Equivalently (pruning rule):

If a subset is **NOT** frequent,
none of its supersets **can** be frequent.

{Coffee, Milk}
not frequent
✕ ------→
{Coffee, Milk, Bread}
(any superset)
cannot be frequent

Example

If {Coffee, Milk} is not frequent, then we do not generate or test:

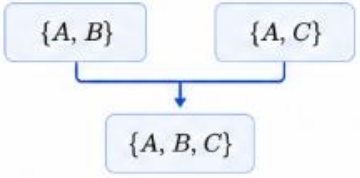
- {Coffee, Milk, Bread}
- {Coffee, Milk, Bread, Sugar}
- … and any larger itemsets containing {Coffee, Milk}

Candidate Generation

We generate the next level of candidates C_{k+1} from the current frequent itemsets L_k.

1 JOIN (Self-Join)

Join pairs of frequent k-itemsets that share the first $k-1$ items to create $(k+1)$-item candidates.



Share first $k-1=1$ item (the item A).
→ Candidate $\{A, B, C\}$

2 REMOVE DUPLICATES

The same candidate may be produced by different pairs. Keep each candidate only once.

$\{A, B, C\}$ $\{A, B, C\}$ $\{A, B, C\}$ → $\{A, B, C\}$

Where we are in Apriori

Generate Candidates C_k → Prune using Apriori Property → Support Counting → Keep Frequent Itemsets L_k

Example: Generate C_3 from L_2

Let

$$L_2 = \{\{A, B\}, \{A, C\}, \{A, D\}, \{B, C\}\}$$

Join step (compatible pairs):

- $\{A, B\} + \{A, C\} \rightarrow \{A, B, C\}$
- $\{A, B\} + \{A, D\} \rightarrow \{A, B, D\}$
- $\{A, C\} + \{A, D\} \rightarrow \{A, C, D\}$

After removing duplicates:

$$C_3 = \{\{A, B, C\}, \{A, B, D\}, \{A, C, D\}\}$$

L_k (frequent k-itemsets) → **JOIN** (Self-Join) → C_{k+1} (raw candidates) → **REMOVE DUPLICATES** → C_{k+1} (unique candidates) → next step: Pruning

Candidate Pruning

After generating candidates C_k (size k), we prune them using the Apriori property to obtain the smaller candidate set before support counting.

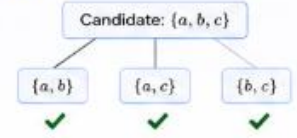
Goal: Remove candidates that cannot be frequent so that we count support for as few itemsets as possible.

Where we are in Apriori

Generate Candidates C_k → **Prune using Apriori Property** → Support Counting → Keep Frequent Itemsets L_k

Several Approaches for Candidate Pruning

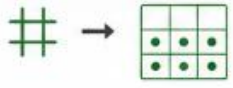
1 Apriori (Subset Checking)



Keep k-item candidate X only if all its $(k-1)$-subsets are frequent (belong to L_{k-1}).

The classic Apriori pruning. Simple and effective.

2 Hash-Based Pruning (DHP)



Hash k-item candidates; eliminate candidates that hash to a bucket with no frequent $(k-1)$-subset.

Prunes many candidates early using hashing.

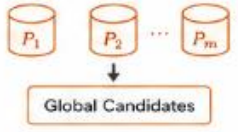
3 Transaction Reduction (Prefix Pruning)



Remove from transactions items that are not frequent (not in L_1) before generating candidates.

Produces smaller transactions and fewer candidates.

4 Partition-Based Pruning



Find frequent itemsets in each partition; combine to form global candidate set.

Reduces candidates by working on smaller partitions.

5 Closed / Maximal-Pattern Pruning (Extension)



Prune candidates that are subsets of already discovered frequent itemsets (keep only closed or maximal candidates).

Reduces number of candidates and final output.

Example: Apriori (Subset Checking)

Suppose $L_2 = \{\{a,b\}, \{a,c\}, \{b,c\}\}$
and candidates $C_3 = \{\{a,b,c\}, \{a,b,d\}, \{a,c,d\}, \{b,c,d\}\}$

Candidate	Its 2-subsets	All in L_2?	Keep?
$\{a,b,c\}$	$\{a,b\}, \{a,c\}, \{b,c\}$	Yes	✓
$\{a,b,d\}$	$\{a,b\}, \{a,d\}, \{b,d\}$	No	✗
$\{a,c,d\}$	$\{a,c\}, \{a,d\}, \{c,d\}$	No	✗
$\{b,c,d\}$	$\{b,c\}, \{b,d\}, \{c,d\}$	No	✗

Result after pruning: $C_3^{\text{pruned}} = \{\{a,b,c\}\}$

Only this candidate needs its support counted.

Comparison on Our Running Example (6 items, minsup = 2)

Method	Idea	Effect on # Candidates	Cost / Overhead	Best When
Subset Checking (Apriori)	Keep X only if all $(k-1)$-subsets of X are frequent	Large reduction, especially as k increases	Need set lookups for each candidate	General use, easy to implement
Hash-Based (DHP)	Eliminate candidates whose hash bucket has no valid subset	Very large reduction early	Extra memory for hash structure	Many candidates, large C_k
Transaction Reduction	Remove infrequent items from transactions	Smaller transactions → fewer candidates	One-time scan per level	Dense transactions with many items
Partition-Based	Mine partitions separately, then combine	Fewer global candidates	Overhead to manage partitions	Very large databases
Closed/Maximal (Extension)	Prune subsets of already frequent itemsets	Much fewer candidates and outputs	More complex management	When only closed or maximal patterns are needed

Support Counting

After generating candidates C_k, we need to count the support of each candidate in the database.

Goal: Compute $support(X)$ for every $X \in C_k$ as efficiently as possible.

Where we are in Apriori

Generate Candidates C_k → Prune using Apriori Property → **Support Counting** → Keep Frequent Itemsets L_k

Several Approaches for Support Counting

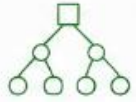
1 Naive Scanning



For each transaction, check every candidate.

Easy to implement, but can be expensive.

2 Hash Tree



Organize candidates in a hash tree; examine only potentially matching candidates.

Reduces number of candidate comparisons.

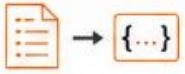
3 Hash Table / Hashing



Hash candidates or their prefixes; use hashing to find and update counts.

Faster lookup and updates.

4 Subset Generation (from Transactions)



Generate all k-subsets of each transaction and look them up in C_k.

Good when $|C_k|$ is large but transactions are short.

5 Trie / Prefix Tree



Store candidates in a trie; share common prefixes to save comparisons.

Useful when many candidates share prefixes.

6 Bit-Vectors / Bitmaps



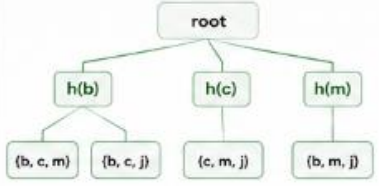
Represent occurrences as bit-vectors and use fast bit operations.

Very efficient for dense datasets.

Example: Hash Tree Approach

Suppose $C_3 = \{\{b, c, m\}, \{b, c, j\}, \{b, m, j\}, \{c, m, j\}\}$
and we process transaction $T = \{b, c, m, n\}$

Hash Tree of Candidates (example)



How it works for $T = \{b, c, m, n\}$

1. Start from root.
2. Follow the branch for item 'b' (since $b \in T$).
3. From there, check only candidates under $h(b)$: $\{b, c, m\}$, $\{b, c, j\}$.
4. Count $\{b, c, m\}$ (all items in T), do not count $\{b, c, j\}$ ($j \notin T$).
5. Also follow 'c' and 'm' branches similarly (only relevant parts are visited).

Hash tree avoids checking all candidates for every transaction.
We only visit branches that can possibly match the transaction.

Comparison on Our Running Example (6 transactions, C_3 above)

Method	Candidate checks for one transaction (approx.)	Total checks for 6 transactions (approx.)	Comments
Naive scanning	4	$4 \times 6 = 24$	Check all 4 candidates for every transaction.
Hash tree	$1-2$	$\sim 6-12$	Visit only relevant branches.
Subset generation	$\binom{\lvert T\rvert}{3}$	Depends on $\lvert T\rvert$	Good when transactions are short.
Trie / prefix tree	$\sim 1-2$	$\sim 6-12$	Benefits from shared prefixes.
Bit-vector / bitmap	Word-level ops.	Very small	Excellent for dense datasets.

Example

minsup = 2

C_k Candidate k-itemsets (generated)
L_k Frequent k-itemsets (after pruning)
➡ Next iteration

Transaction Database

TID	Items
1	b, c, j, m
2	b, c, m, p
3	b, j, m, n
4	b, c, m
5	c, j, m
6	b, c, j, m

Brute-force search vs Apriori

Candidate–transaction checks
378 → 78
78.6% fewer operations

Itemsets requiring support counting
63 → 13
79.4% fewer itemsets

Candidate memory (counters)
63 → 6
90.5% less candidate memory

How to read:

- Generate candidates C_k
- Count support of each candidate
- Prune (keep only those with support $\geq minsup$) → L_k
- Use L_k to generate next level candidates C_{k+1}

1 $k = 1$

1. Generate Candidates (C_1) — All 1-itemsets

Itemset	Support
{b}	5
{c}	5
{j}	4
{m}	5
{n}	1
{p}	1

➡ 2. Count Support — Scan DB and count support of each candidate.

➡ 3. Prune (support ≥ 2) — Keep only frequent 1-itemsets

Itemset	Support
{b}	5
{c}	5
{j}	4
{m}	5

➡ L_1 (Frequent 1-itemsets): $\{b\}, \{c\}, \{j\}, \{m\}$

2 $k = 2$

1. Generate Candidates (C_2) — All 2-itemsets from L_1

Itemset	Support
{b, c}	4
{b, j}	3
{b, m}	4
{c, j}	3
{c, m}	4
{j, m}	2

➡ 2. Count Support — Scan DB and count support of each candidate.

➡ 3. Prune (support ≥ 2) — Keep only frequent 2-itemsets

Itemset	Support
{b, m}	4
{b, c}	4
{c, m}	4
{c, j}	3

➡ L_2 (Frequent 2-itemsets): $\{b, m\}, \{b, c\}, \{c, m\}, \{c, j\}$

3 $k = 3$

1. Generate Candidates (raw C_3) — All 3-itemsets from L_2

Itemset
{b, c, m}
{b, c, j}
{b, m, j}
{c, m, j}

➡ 2. Apriori Prune — Keep candidates whose all 2-item subsets are in L_2

Itemset	All 2-subsets frequent?
{b, c, m}	Yes
{b, c, j}	No {b, j} ✗
{b, m, j}	No {b, j}, {j, m} ✗
{c, m, j}	No {m, j} ✗

➡ 3. Count Support — Only remaining candidates are counted.

Itemset	Support
{b, c, m}	3

➡ 4. Prune (support ≥ 2) — Keep only frequent 3-itemsets

Itemset	Support
{b, c, m}	3

➡ L_3 (Frequent 3-itemsets): $\{b, c, m\}$

⭐ **Result:** Union of all L_k gives all frequent itemsets.

$L_1 = \{b\}, \{c\}, \{j\}, \{m\}$ $L_2 = \{b, m\}, \{b, c\}, \{c, m\}, \{c, j\}$ $L_3 = \{b, c, m\}$

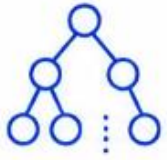
Apriori Limitations



1 Repeated database scans

Apriori scans the entire database once for each candidate set C_k to count support.

→ Many passes over the data (up to max length L).



2 Candidate explosion

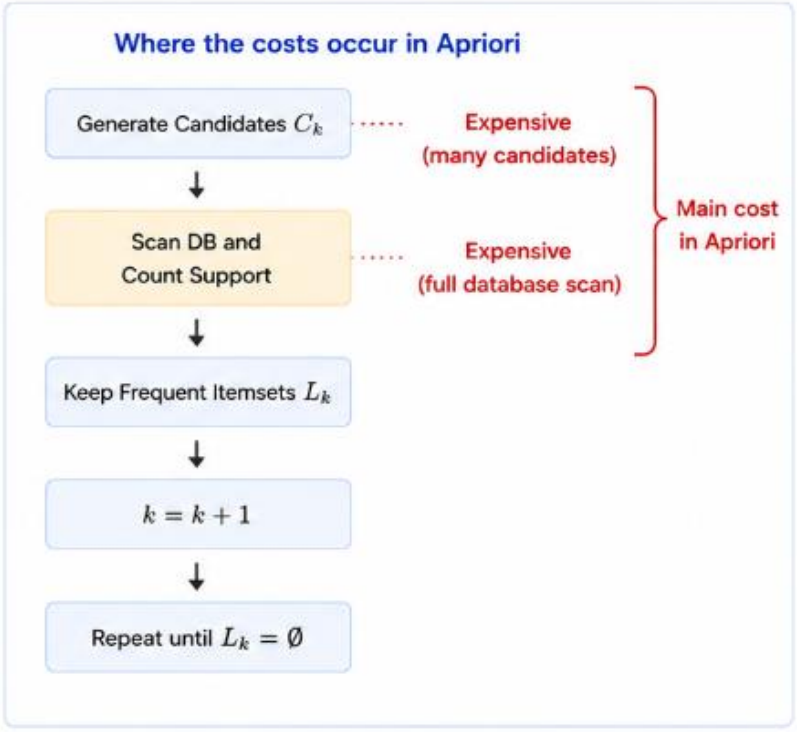
The number of candidate k-itemsets $|C_k|$ can grow exponentially with k and the number of items.

→ Many candidates are generated only to be pruned later.



3 Expensive candidate generation and support counting

Generating candidates and counting their support require significant CPU time and memory, especially for large k.



FP-Growth: An Alternative to Apriori

FP-Growth was introduced by Jiawei Han, Jian Pei and Yiwen Yin in 2000 (SIGMOD paper: "Mining Frequent Patterns without Candidate Generation").

Key idea: Mine frequent patterns **without** generating candidate itemsets.

1 Count frequent items

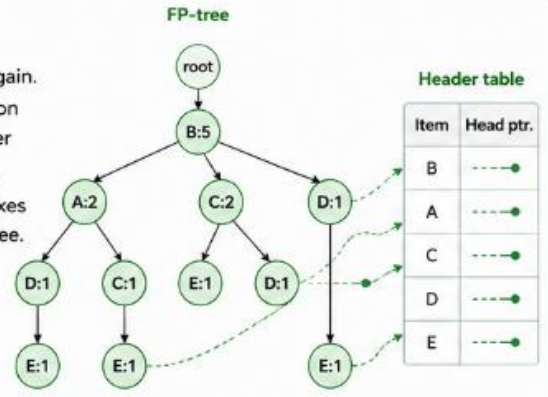
- Scan the database once.
- Remove infrequent items.
- Order remaining items by descending frequency.

Item	B	A	C	D	E
Support	5	4	4	3	3

(Minimum support = 3)

2 Build FP-tree

- Scan the database again.
- Insert each transaction using the global order (B > A > C > D > E).
- Share common prefixes to build a compact tree.



3 Mine the FP-tree

- Process items from least frequent to most frequent (E → D → C → A → B).
- For each item, build its conditional pattern base.
- Recursively build conditional FP-trees and grow patterns.

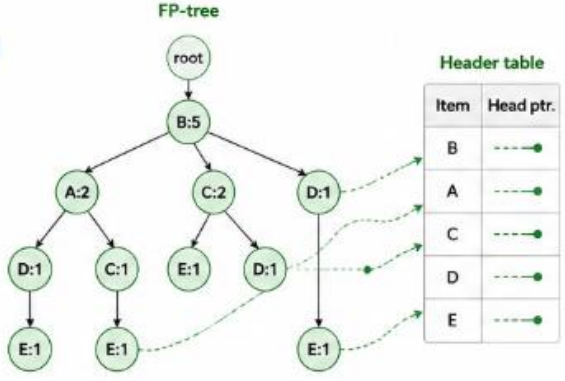
Frequent Itemsets

Example Transactions

TID	Items
T1	A, B, D, E
T2	B, C, E
T3	A, B, C, E
T4	B, C, D
T5	B, D, E

After removing infrequent items and ordering by frequency (B > A > C > D > E)

TID	Ordered Items
T1	B, A, D, E
T2	B, C, E
T3	B, A, C, E
T4	B, C, D
T5	B, D, E



Mining (example)

- Start with E (least frequent).
- Conditional pattern base of E: {B,A,D:1}, {B,C:1}, {B,A,C:1}, {B,D:1}, {B:1}
- Build conditional FP-tree for E, mine patterns with E.
- Repeat for D, C, A, B.

▶ All frequent itemsets are generated without candidate generation.

Building the FP-tree

1 Example transactions

TID	Items
T1	A, B, D, E
T2	B, C, E
T3	A, B, C, E
T4	B, C, D
T5	B, D, E

Minimum support = 2
(all items are frequent)

2 Count item frequencies

Item	Support count
B	5
E	4
C	3
D	3
A	2

All items are frequent
(support ≥ 2)

3 Global order (high → low)

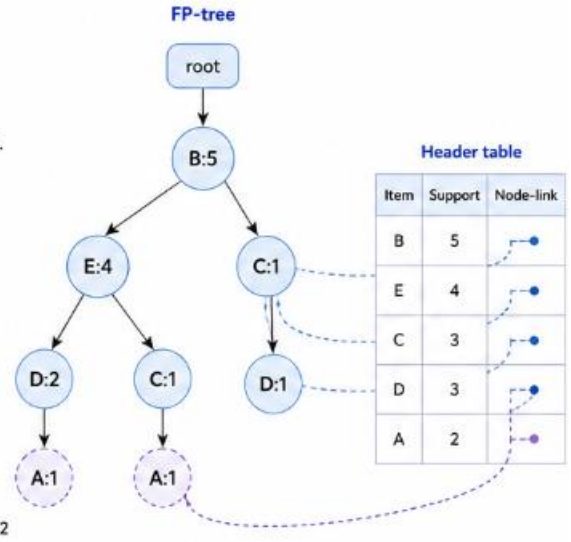
B > E > C > D > A
(C before D to break tie)

4 Order items in each transaction according to the global order

TID	Ordered items
T1	B, E, D, A
T2	B, E, C
T3	B, E, C, A
T4	B, C, D
T5	B, E, D

5 Insert ordered transactions into the FP-tree

1. Insert T1: B, E, D, A
   Create path
   B → E → D → A
2. Insert T2: B, E, C
   From root follow B → E.
   Create/extend path
   B → E → C
3. Insert T3: B, E, C, A
   Follow B → E → C.
   Add A under C
4. Insert T4: B, C, D
   From root follow B.
   Create/extend path
   B → C → D
5. Insert T5: B, E, D
   Follow B → E → D.
   Increase count of D: 1 → 2



Key points

- Items are ordered once using global frequencies.
- Each transaction is rearranged according to this order.
- Transactions sharing the same prefixes share the same path in the tree.
- Counts on nodes show how many transactions pass through that prefix.
- The tree is a compact representation of the database.

Item:count node
Node-link (same item)
Single occurrence

Result

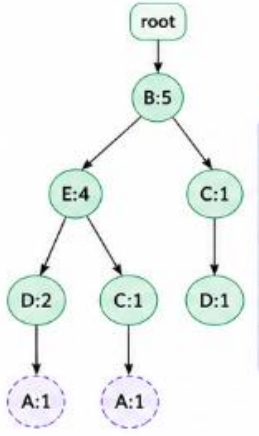
The FP-tree and header table are built using only two scans of the database:
1) count item frequencies
2) build the tree

This structure is then used to mine frequent patterns without generating candidate itemsets.

Mining the FP-tree

We process items in **ascending frequency** (from least frequent to most frequent).
For each item we build its **conditional pattern base**, the conditional FP-tree, and mine frequent patterns.

FP-tree from previous slide



Header table

Item	Support
B	5
E	4
C	3
D	3
A	2

Minimum support = 2

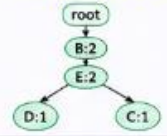
Process items in ascending frequency: A (2) → D (3) → C (3) → E (4) → B (5)

1 Item: A (support = 2)

Conditional pattern base

Prefix path	Count
B, E, D	1
B, E, C	1

Conditional FP-tree

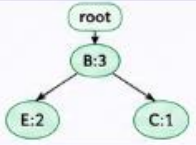


Frequent itemsets with A
{A} (2)
{A, B} (2)
{A, E} (2)
{A, B, E} (2)

2 Item: D (support = 3)

Conditional pattern base

Prefil	Count
B, E	2
B, C	1

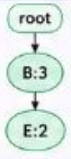


Frequent itemsets with D
{D} (3)
{B, D} (3)
{D, E} (2)
{B, D, E} (2)

3 Item: C (support = 3)

Conditional pattern base

Prefil	Count
B, E	1
B, E	1
B	1

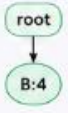


Frequent itemsets with C
{C} (3)
{B, C} (3)
{C, E} (2)
{B, C, E} (2)

4 Item: E (support = 4)

Conditional pattern base

B	Count
B	2
B	1
B	1



Frequent itemsets with E
{E} (4)
{B, E} (4)

5 Item: B (support = 5)

No prefix paths
(B is the most frequent item)

Frequent itemsets with B
{B} (5)

Result: Frequent itemsets (minsup = 2)

Itemset	Support
{A}	2
{B}	5
{C}	3
{D}	3
{E}	4
{A, B}	2
{A, E}	2
{B, C}	3
{B, D}	3
{B, E}	4
{C, E}	2
{D, E}	2
{A, B, E}	2
{B, C, E}	2
{B, D, E}	2

Total: 15 frequent itemsets

How it works

- Start from the least frequent item and move upward.
- For each item, use its prefix paths to build a smaller (conditional) FP-tree.
- Mine all frequent patterns that include the current item.
- No candidate itemsets are generated.

Key advantages

- ✓ Only two scans of the database (count + build tree).
- ✓ Compact tree representation.
- ✓ Efficient for large and dense datasets.

Next step: Association rules

Example from the results:

- Rule E → B : confidence = 4/4 = 100%
- Rule B → E : confidence = 4/5 = 80%

(Rules are generated from the frequent itemsets.)

Ecalt

Eclat (Equivalence Class Transformation), introduced by Mohammed J. Zaki in 2000, is a depth-first algorithm for frequent itemset mining. Instead of counting item occurrences in transactions, Eclat uses a vertical representation: for each item, it stores the set of transaction IDs (TIDs) containing it.

1) Horizontal database (transactions)

TID	Items
T1	A, B, D, E
T2	B, C, E
T3	A, B, C, E
T4	B, C, D
T5	B, D, E

Transform to vertical representation →

2) Vertical representation (TID sets)

Item	TID set (transactions containing the item)
A	{T1, T3}
B	{T1, T2, T3, T4, T5}
C	{T2, T3, T4}
D	{T1, T4, T5}
E	{T1, T2, T3, T5}

Support by set intersection →

3) Support of an itemset = size of intersection of TID sets

Example: {A, E}

T(A) = {T1, T3}

T(E) = {T1, T2, T3, T5}

T(A ∩ E) = {T1, T3} → support = 2

Example: {B, C, E}

T(B) = {T1, T2, T3, T4, T5}

T(C) = {T2, T3, T4}

T(E) = {T1, T2, T3, T5}

T(B ∩ C ∩ E) = {T2, T3} → support = 2

Example

Goal: Find all frequent 2-itemsets using Eclat.
We intersect TID sets. **Support** of {X, Y} = |T(X) ∩ T(Y)|. Minimum support (minsup) = 2 (count)

1) Vertical representation (TID sets)

Item	TID set (transactions containing the item)
A	{T1, T3}
B	{T1, T2, T3, T4, T5}
C	{T2, T3, T4}
D	{T1, T4, T5}
E	{T1, T2, T3, T5}

We process items in alphabetical order:
A → B → C → D → E

2) Compute 2-itemsets by intersecting TID sets

Base item	Pair	Intersection (TID set)	Support (count)	Frequent? (minsup = 2)
A	A, B	{T1, T3}	2	Yes
	A, C	{T3}	1	No
	A, D	{T1}	1	No
	A, E	{T1, T3}	2	Yes
B	B, C	{T2, T3, T4}	3	Yes
	B, D	{T1, T4, T5}	3	Yes
	B, E	{T1, T2, T3, T5}	4	Yes
C	C, D	{T2, T4}	2	Yes
	C, E	{T2, T3}	2	Yes
D	D, E	{T1, T5}	2	Yes

Example computations

T(A) = {T1, T3}
T(B) = {T1, T2, T3, T4, T5}
T(A) ∩ T(B) = {T1, T3}
support({A, B}) = 2 ✓

T(B) = {T1, T2, T3, T4, T5}
T(C) = {T2, T3, T4}
T(B) ∩ T(C) = {T2, T3, T4}
support({B, C}) = 3 ✓

T(A) = {T1, T3}
T(D) = {T1, T4, T5}
T(A) ∩ T(D) = {T1}
support({A, D}) = 1 ✗

Key idea

- Eclat finds 2-itemsets by intersecting TID sets.
- The size of the intersection is the support count.
- If support < minsup, the itemset is not frequent and that branch is pruned.

Frequent 2-itemsets (minsup = 2)

{A, B}, {A, E}, {B, C}, {B, D}, {B, E},
{C, D}, {C, E}, {D, E} → 8 frequent 2-itemsets

Example

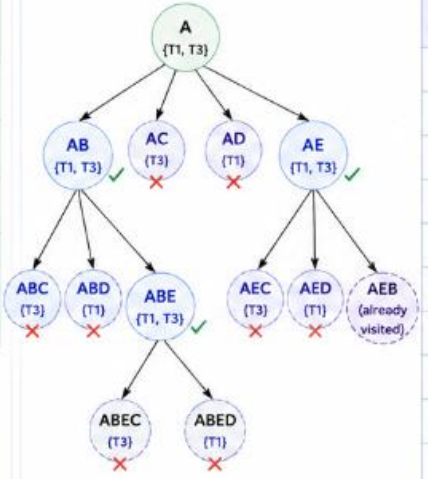
Eclat grows itemsets **depth-first.**
To extend a frequent itemset by a new item, intersect their TID sets.
If the intersection size (support count) is ≥ **minsup**, keep it and continue; otherwise, **prune** that branch.

1) Vertical representation (TID sets)

Item	TID set (transactions containing the item)
A	{T1, T3}
B	{T1, T2, T3, T4, T5}
C	{T2, T3, T4}
D	{T1, T4, T5}
E	{T1, T2, T3, T5}

Minsup = 2 (count)

2) Depth-first growth using intersections



support ≥ 2 (frequent)
support < 2 (pruned)

Itemset	TID set (intersection)	Support (count)	Frequent? (minsup = 2)
A	{T1, T3}	2	✓
AB	{T1, T3}	2	✓
ABC	{T3}	1	✗
ABD	{T1}	1	✗
ABE	{T1, T3}	2	✓
ABEC	{T3}	1	✗
ABED	{T1}	1	✗
AC	{T3}	1	✗
AD	{T1}	1	✗
AE	{T1, T3}	2	✓
AEC	{T3}	1	✗
AED	{T1}	1	✗

How to read the example

- Start with item A.
- Intersect its TID set with items that appear after A in the order **A → B → C → D → E.**
- Keep only itemsets with support ≥ 2 and continue growing them.
- Stop (prune) a branch when support < 2.

Example: grow AB

T(AB) = T(A) ∩ T(B) = {T1, T3} → 2 ✓

T(ABC) = T(AB) ∩ T(C) =
{T1, T3} ∩ {T2, T3, T4} = {T3} → 1 ✗

T(ABE) = T(AB) ∩ T(E) =
{T1, T3} ∩ {T1, T2, T3, T5} = {T1, T3} → 2 ✓

Key idea

- Eclat extends frequent itemsets by intersecting TID sets.
- The intersection size is the support count.
- Depth-first search explores deep and prunes early.

Frequent itemsets discovered (minsup = 2)

1-itemsets: {A}, {B}, {C}, {D}, {E}
2-itemsets: {A, B}, {A, E}, {B, C}, {B, D}, {B, E}, {C, D}, {C, E}, {D, E}
3-itemsets: {A, B, E}, {B, C, E}, {B, D, E}

Total: 15 frequent itemsets

From Rules to Knowledge: Interpreting Results



1. Evaluating Association Rules

For a rule $X \rightarrow Y$

Support	: How common is $X \cup Y$?	$P(X \cup Y)$
Confidence	: When X occurs, how often does Y occur?	$P(Y\|X)$
Lift	: Is Y more likely with X than normally?	$\frac{P(Y\|X)}{P(Y)}$
Interest	: How much does $P(Y\|X)$ differ from $P(Y)$?	$P(Y\|X) - P(Y)$

No single measure is best – use several together.

2. Too Many Rules: Manage the Explosion

- Low support thresholds can produce thousands of rules that are hard to review.
- Common practice:

1. Set thresholds (e.g., minsup, minconf, minlift)
2. Rank rules (by lift, confidence, conviction, etc.)
3. Remove redundant rules
4. Apply domain constraints
5. Expert evaluation & actionability

The goal is quality over quantity.

3. Redundant Rules

- More specific rules may provide little new information.
- Example:

Rule	Support	Confidence	Lift
{Diabetes} → {DrugA}	0.12	0.82	1.35
{Diabetes, Age>65} → {DrugA}	0.06	0.83	1.37

Adding "Age>65" barely changes confidence or lift → the second rule may be redundant.

Prefer simpler, more general rules.

4. Association ≠ Causation

$X \rightarrow Y$ (Association)

X causes Y (Causation)



- Association rules show co-occurrence, not cause and effect.
- Always consider alternative explanations (confounders, data collection process, seasonality, policies, etc.).

Do not make causal decisions based only on association rules.

5. Simpson's Paradox: Associations Can Be Misleading

A trend in the overall data can disappear or even reverse in subgroups.

Overall (All Patients)

Treatment	Recovered	Recovery Rate
Drug A	700 / 1000	70%
Drug B	600 / 1000	60%

Overall conclusion: Drug A looks better.

But when we stratify by disease severity:

Severity	Drug A		Drug B	
	Recovered / Total	Rate	Recovered / Total	Rate
Mild	450 / 500	90%	475 / 500	95%
Severe	250 / 500	50%	125 / 500	25%

Drug B is better in both groups, but looks worse overall.
Why? Because Drug A was used more for mild cases.

Factors Affecting Complexity of Frequent Itemset Mining

